PLA Algorithm Report

Matt Vlodek and Noah Vlodek

10 September 2024

1 PLA Implementation

1.1 Generating Data Points

1.1.1 Case 1: Linearly Separable Training Data

The first thing we did is create 30 random x-values. We used the *np.random.randint* method from the Numpy library to accomplish this task. Next, we created a line in slope-intercept form as a guide to formulate the data. We ensured that 15 points were randomly generated above the line, and 15 points were randomly generated below the line, ensuring that the data is linearly separable with even classes.

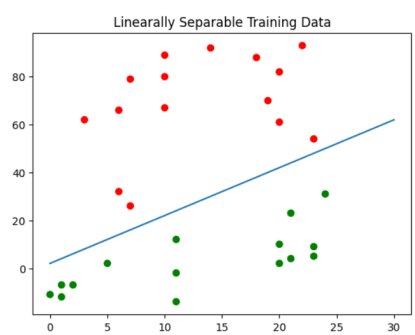

Figure 1: Linearly Separable Graph

1.1.2 Case 2: Non-Linearly Separable Training Data

For this case, we used the same x-values from the first case, shuffling them to induce pseudo-randomness. Then, we created 30 y-values in the same manner as we generated the x-values in case 1. We combined the x-values and y-values in an embedded list containing the data points. To assign classes, we navigated the data, and randomly assigned classes using *np.random.choice*. We added checks to ensure that each class had no more than 15 data points. Finally, we organized the data with classes in a DataFrame for organization.

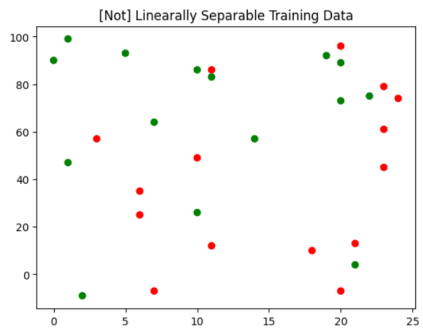

Figure 2: Non-Linearly Separable Data

1.1.3 Case 3: Test Data

We followed a similar procedure as we had for the training data. We generated 10 random x-values and created a line to formulate the data. We randomized 5 points above the line and 5 points below to form a **linearly separable** dataset. Again, we assigned these points proper class distinction and visualized the points in a data frame.

1.2 Run of the Code: Linearly Separable Training Data

Weights and Constants The initial weights were chosen by randomly generating them through 1 and 20 using the *np.random.randint* function. Then, as we iterated through the points, updated the weights in effort to find a line that separates the data by classes.

Final Solution Equation of Line

$$2 - 82x_1 + 46x_2 = 0 \tag{1}$$

Total Number of Weight Vector Updates The total number of weight vector updates when running the code on linearly separable data was **8**.

Total Number of Iterations over Training Set The total number of iterations over the training set during this time was **4**.

Final Misclassification Error To calculate the misclassification error, we created a contingency in our PLA class that is executed on only the final iteration. We created an empty list to store the final predictions for the classes, then compared those predictions to the actual classes of each point. This gave us the

total number of misclassifications for the dataset. Then, we divided the number of misclassifications by the total number of predictions in the last iteration. This gave us the misclassification error.

$$MisclassificationErr = \frac{NumMisclassifications}{NumPredictions}$$

Data	Misclassification Error
Training	0.0%
Testing	10.0%

1.3 Run of the Code: Non-Linearly Separable Training Data

Weights and Constants The initial weights were chosen by randomly generating them through 1 and 20 using the *np.random.randint* function. Then, as we iterated through the points, updated the weights in effort to find a line that separates the data by classes.

Final Solution Equation of the Line

$$-35 + 100x_1 - 50x_2 = 0 \tag{2}$$

Total Number of Weight Vector Updates The total number of weight vector updates when running the code was **1059**.

Total Number of Iterations over Training Set The total number of iterations over the training set during this run was **82**.

Final Misclassification Error

Data	Misclassification Error
Training	20.0%
Testing	60.0%

1.4 Rerunning the Code for Different Parameters

1.4.1 Linearly Separable Training Data

Varying the Initial Weights We first generated random weights using the *numpy.random.randint* module for the linearly separable training data. We updated this to use the *numpy.random.uniform* module to create random float values for the weights. The updated solution contained more precise values for the weights. We rounded the solution to 5 decimal places for readability. The model found the solution within 2 iterations through the training set for this run, which is an improvement from the 4-5 iterations it took for integer weights. The classification error on the training data was unchanged, at zero percent.

The misclassification error on the testing data ranges from 0 to 20 percent, a slight improvement as well.

The line of code we modified became this:

```
# Try random floating point values for weights
    self.weights = np.random.uniform(1.0, 20.0, size=dim)
```

Final Solution Equation of Line:

$$3.03444 - 18.54532x_1 + 10.48469x_2 = 0 \tag{3}$$

When we used ones for the initial weights, using *np.ones*, the final solution of the equation did not change much. But, the misclassification error went up to 20 percent every time for the testing data. There were only 5 weight vector updates for this iteration, a mild decrease.

Varying the Step Size Constant For this part, we varied the values of the learning rate in our PLA class. We tried different values between 0 and 1. Below, I will display the results of the trials. As we decreased the learning parameters, the coefficients of x1 and x2 seem to decrease in magnitude as well. When the learning parameter is large, the coefficients of x1 and x2 are large -294, 225, and when we decrease the learning parameter to 0.05, the values (magnitude) of x1 and x2 are smaller.

Learning Rate (η)	Equation of Line	Updates	Iterations	($\mathrm{Trn}_E rr, Tst_E rr$)
1.0	-10 - 294x$_1$ + $225x_2 = 0$	8	4	(0, 20)
0.75	3 - 61x$_1$ + $34.75x_2 = 0$	8	4	(0,10)
0.5	5 + 10x$_1$ + $4.5x_2 = 0$	2	2	(0,0)
0.25	5 - 19.25x$_1$ + $11.0x_2 = 0$	8	4	(0,10)
0.1	5.3000 - 7.8000x$_1$ + $5.699x_2 = 0$	11	4	(0,20)
0.05	5.000 + 3.0x$_1$ + $2.05x_2 = 0$	12	3	(0,20)

Table 1: PLA Iterations with Varying η

Varying the Order in Considering Training Points For the linear case, when we shuffled the indices after each shuffle the equation for the line increased in slope. When we went through the training points in an unspecified order, the slope jumped from 0.56 to 2.02, more than doubling. This line seemed to work better for unseen data as well: the testing classification error rate fell from 10-percent to 0-percent.

$$-85x_1 + 42x_2 = 0 \tag{4}$$

1.4.2 Non-Linearly Separable Training Data

Varying the Initial Weights We first generated random weights using the *numpy.random.randint* module for training data that is not linearly separable.

We updated this to use the *numpy.random.uniform* module to create random float values for the weights. Once again, the model failed to find a solution and it took 20 iterations for the model to give up on finding a solution due to stagnation. The final misclassification error was 20%.

Varying the Step Size Constant For this part, we varied the values of the learning rate in our PLA class. We tried different values between 0 and 1. Below, I will display the results of the trials. As we decreased the learning parameters, the coefficients of x1 and x2 seem to decrease in magnitude as well. The data was not linearly separable so the PLA algorithm did not find a solution during any of the iterations. Additionally, all the algorithms completed after 40 or fewer iterations due to stagnation in the misclassification errors. When we updated the learning rate to be 0.5, the algorithm executed the fewest iterations, finding a stagnation error quickly, with a relatively low training error at 13.3%. Below is a table displaying the final equations of the lines, although these do **not** represent ***solutions***. Also noted is the number of weight updates, iterations, and the error on the test and training sets.

Learning Rate (η)	Equation of Line	Updates	Iterations	($\mathrm{Trn}_E rr, Tst_E rr$)
1.0	17.0 - 15.0x$_1$ $- 33.0x_2 = 0$	165	30	(16.7, 60.0)
0.75	12.0 - 6.0x$_1$ $- 31.75x_2 = 0$	108	20	(13.3,60.0)
0.5	11.5 - 21.5x$_2$ $= 0$	51	10	(13.3,60.0)
0.25	12.5 + 1.25x$_1$ $- 15.5x_2 = 0$	116	20	(23.3,60.0)
0.1	11.4000 - 0.8000x$_1$ $- 3.200x_2 = 0$	106	20	(20.0,60.0)
0.05	11.65 - 0.70x$_1$ $- 1.40x_2 = 0$	229	40	(20.0,60.0)

Table 2: PLA Iterations with Varying η

Varying the Order in Considering Training Points Shuffling the data for the nonlinear case created a final equation that was almost double the slope and it also cut the test error down by 10-percent (60 -¿ 50). So, overall shuffling the data points results in a better classifier for unseen data.

$$1.85x_1 - x_2 - 0.448 = 0 \tag{5}$$

2 Source Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy as sci

class PLA:
    # Constructor initializing learning_rate, max iterations and weights
    def __init__(self, learning_rate=1.0, max_iters=1000, tolerance_iters = 10):
```

```
        self.learning_rate = learning_rate
        self.max_iters = max_iters
        self.weights = None
        self.iters = 0
        self.wvecupdates = 0
        self.misclass = 0
        self.tolerance_iters = tolerance_iters
        self.total_predictions = 0


    # Augmenting a 1 at the beginning of each sublist in the list
    def augment_x(self, x):
        return np.array([np.insert(point, 0, 1) for point in x])

    def predict(self, x):
        # Predict the sign of the class for each point
        pred_y = np.dot(self.weights, x)

        return 1 if pred_y >= 0 else -1

    # Implement PLA where x - data points and y - class
    def fit(self, x, y):
        # Augment the vectors with 1
        x = self.augment_x(x)
        # Use 'self.augment_x()' as it's a method of the class

        dim = len(x[0])

        # Initialize d + 1 weights for the line w0x0 + w1x1 + ..... + wdxd
        self.weights = np.random.randint(1, 20, size=dim, dtype = int)

        # List to store final predictions
        final_iter_preds = []
        misclass = []

        # Navigate through training set a predetermined number of times
        for _ in range(self.max_iters):
            self.iters += 1
            # Set update initially to false
            updated = False


            # Go through all of the points. y holds the actual class
            for j, k in zip(x, y):
                pred_class = []
```

```
                # Predict the class based on point
                pred_y = self.predict(j)

                pred_class.append(pred_y)

                if self.iters == self.max_iters:
                    final_iter_preds.append(pred_y)

                if pred_y * k < 0:
                    # Misclassification -> update weights
                    self.misclass += 1
                    self.weights = self.weights + self.learning_rate * k * j
                    updated = True
                    # Increment weight vector updates
                    self.wvecupdates += 1

            # Check to see if there was a stagnation after
            # the tolerance iters
            misclass.append(self.misclass / len(x))
            if len(misclass) == self.tolerance_iters:
                if misclass[0] == misclass[len(misclass) - 1]:
                    print(f'At iteration: {self.iters} there was a stagnation')
                    print(f'Final Misclassification Error: {misclass[0]}')
                    break
                else:
                    misclass = []

            # Check to see if we updated the points
            if not updated:
                break

            self.misclass = 0
            self.total_predictions = len(x)
            for pred, actual in zip(final_iter_preds, y):
                if pred * actual < 0:
                    self.misclass += 1

        misclass_error = (self.misclass / self.total_predictions) * 100
        print(f'Misclassification error: {misclass_error}%')

        print(f'Weights:', f'{self.weights}')
        print(f'Equation of the line: {self.weights[0]} +
            {self.weights[1]}x + {self.weights[2]}y = 0')


        # Check to see if we updated the weights
```

```
        if not updated:
            print('Very first solution found')
        else:
            print('Either no solution was found or a solution was found
            but not verified in the last pass.')

    def get_iters(self):
        return self.iters

    def get_weight_updates(self):
        return self.wvecupdates

    def get_misclassifications(self):
        if self.total_predictions > 0:
            return (self.misclass/self.total_predictions)
        return 0.0

    def get_weights(self):
        return self.weights

# Set random seed
np.random.seed(42)
# Make sure there are 30 points
num_points = 30

# x for case 1 and case 2
x_cases = np.random.randint(low = 0, high = 25, size = num_points, dtype = int)
print(x_cases)

print(len(x_cases))

# Line that points will be above and below
x = np.linspace(0, 30, num = 50)
m, b = 2, 2
y_line = m*x + b

data_points = []
# Generate 15 points above the line and 15 points below the line

for i, x_val in enumerate(x_cases):
    y_on_line = m * x_val + b

    if i < num_points // 2:
        y_high = np.random.randint(low = y_on_line + b + 2,
        high = 100, size = 1, dtype = int)
```

```
        # Append the x value, y-value and class '+1'
        data_points.append((x_val, y_high[0], 1))
    else:
        y_low = np.random.randint(low = -15, high = y_on_line - 2, size = 1,
        dtype = int)
        # Append the x value, y-value, and class '-1'
        data_points.append((x_val, y_low[0], -1))
np.random.seed(44)

# Use x_cases from earlier as the x values
# Generate random y_values
y_case2 = np.random.randint(low = -10, high = 100, size = num_points,
dtype = int)

x_case2 = x_cases.copy()
np.random.shuffle(x_case2)


dp_case2 = []
for x, y in zip(x_case2, y_case2):
    dp_case2.append([x, y])

class_1 = 0
class_2 = 0
# Assign a random class
for i, dp in enumerate(dp_case2):

    # Check if class_1 has 15 points
    if class_1 == 15:
        dp_case2[i].append(1)
        class_2 += 1
    # Check if class_2 has 15 points
    elif class_2 == 15:
        dp_case2[i].append(-1)
        class_1 += 1
    # Enters if both classes are not full (have 15 data points)
    else:
        op = [-1, 1]
        cl = np.random.choice(op, size = 1)
        dp_case2[i].append(cl[0])

        if cl == 1:
            class_2 += 1
        else:
            class_1 += 1
```

```
x_points = []
y_points = []
classes = []
for point in dp_case2:
    x_points.append(point[0])
    y_points.append(point[1])
    classes.append(point[2])

data2 = {'X': x_points, 'Y': y_points, 'Class': classes}

df2 = pd.DataFrame(data2)
df2

colors = ['red' if i == 1 else 'green' for i in df['Class']]

plt.scatter(x_points, y_points, color = colors)
plt.title('[Not] Linearally Separable Training Data')

np.random.seed(42)
# Make sure there are 10 points
num_points = 10

# x for case 1 and case 2
x_test = np.random.randint(low = 0, high = 25, size = num_points, dtype = int)

print(len(x_test))

# Line that points will be above and below
x = np.linspace(0, 30, num = 50)
m, b = 2, 2
y_line = m*x + b

test_points = []
# Generate 5 points above the line and 5 points below the line

for i, x_val in enumerate(x_test):
    y_on_line = m * x_val + b

    if i < num_points // 2:
        y_high = np.random.randint(low = y_on_line + b + 2, high = 100,
        size = 1, dtype = int)
        # Append the x value, y-value and class '+1'
        test_points.append((x_val, y_high[0], 1))
    else:
        y_low = np.random.randint(low = -15, high = y_on_line - 2, size = 1,
        dtype = int)
```

```
        # Append the x value, y-value, and class '-1'
        test_points.append((x_val, y_low[0], -1))

train_lin_sep = PLA()

x_lin = df[['X', 'Y']].values
y_lin = df['Class'].values

weights = train_lin_sep.fit(x_lin,y_lin)

w_updates = train_lin_sep.get_weight_updates()
epochs_tlin = train_lin_sep.get_iters()

print(f'Weight updates: {w_updates}')
print(f'Number of iterations through training set: {epochs_tlin}')


test_points = test_df[['X', 'Y']].values

# Augment the test_points
test_points = [np.insert(point, 0, 1) for point in test_points]

test_classes = test_df['Class'].values

test_ldata_pred_classes =  [train_lin_sep.predict(x) for x in test_points]

misclass = 0
for actual, pred in zip(test_classes, test_ldata_pred_classes):
    if pred != actual:
        misclass += 1
print(f'Misclassification error: {misclass / len(test_points) * 100}%')
print(f'Final Weights: {train_lin_sep.get_weights()}')

train_non_sep = PLA()

x_nlin = df2[['X', 'Y']].values
y_nlin = df['Class'].values

train_non_sep.fit(x_nlin, y_nlin)

w_nupdates = train_non_sep.get_weight_updates()
epochs_ntlin = train_non_sep.get_iters()
non_sep_misclass = train_non_sep.get_misclassifications()


print(f'Weight updates: {w_nupdates}')
```

```
print(f'Number of iterations through training set: {epochs_ntlin}')

test_points = test_df[['X', 'Y']].values

# Augment the test_points
test_points = [np.insert(point, 0, 1) for point in test_points]

test_classes = test_df['Class'].values

test_ldata_pred_classes =  [train_non_sep.predict(x) for x in test_points]

misclass = 0
for actual, pred in zip(test_classes, test_ldata_pred_classes):
    if pred != actual:
        misclass += 1
print(f'Misclassification error: {misclass / len(test_points) * 100}%')
```

3 Training and Testing Data

3.1 Linearly Separable Training Data

X	Y	Class
6	66	1
19	70	1
14	92	1
10	67	1
7	79	1
20	61	1
6	32	1
18	88	1
22	93	1
10	80	1
10	89	1
23	54	1
20	82	1
3	62	1
7	26	1
23	5	-1
2	-7	-1
21	23	-1
20	2	-1
1	-12	-1
23	9	-1
11	-2	-1
5	2	-1
1	-7	-1
20	10	-1
0	-11	-1
11	-14	-1
21	4	-1
11	12	-1
24	31	-1

3.2 Non-Linearly Separable Training Data

X	Y	Class
18	10	1
6	25	1
6	35	-1
10	49	1
20	-7	1
11	86	1
24	74	-1
7	7	-1
20	96	1
21	13	1
23	45	1
11	12	1
23	79	1
3	57	1
23	61	1
14	57	1
20	89	-1
19	-2	-1
1	99	1
0	90	1
1	47	-1
2	-9	-1
21	4	-1
5	93	-1
10	26	-1
11	83	-1
22	75	-1
20	73	-1
10	86	-1
7	64	-1

3.3 Testing Data

X	Y	Class
6	92	1
19	67	1
14	57	1
10	28	1
7	41	1
20	37	-1
6	-14	-1
18	8	-1
22	28	-1
10	14	-1

We certify that this report is our own, independent work and that it does not plagiarize, in part or in full, any other work.